

Redis — Complete Practical Notes

A beginner-to-intermediate guide for backend development, caching, queues, pub/sub, and event-driven systems

1. What is Redis?

Redis (Remote Dictionary Server) is an in-memory data store. It keeps frequently accessed data primarily in RAM, which makes read/write operations extremely fast. Redis is commonly used as a cache, session store, message broker, rate limiter, leaderboard, distributed lock, and queue/event-processing component.

Think of Redis as a very fast shared data structure that multiple applications can access over a network.

2. Why is Redis Fast?

Traditional databases commonly need disk I/O for many operations. Redis keeps its working dataset in memory, so common operations avoid the latency of disk access. Redis also uses efficient data structures and a simple command model. Persistence can still be enabled when durability is required.

Important: Redis being in-memory does **not** mean it must lose everything when the process stops. Redis supports persistence mechanisms such as RDB snapshots and AOF.

3. Core Redis Data Types

Type	What it stores	Typical use
String	Text, numbers, serialized values	Cache, counters, tokens
Hash	Field-value pairs	User/session objects
List	Ordered sequence	Queues, recent items
Set	Unique unordered values	Tags, membership
Sorted Set	Unique values + scores	Leaderboards, ranking
Stream	Append-only event records	Event processing
Bitmap	Bit-level values	Flags, activity tracking

4. Essential Commands

Strings

```
SET name "Krishna"  
GET name  
SET counter 0  
INCR counter  
DECR counter  
DEL name
```

Expiration / TTL

```
SET otp:1234 "4821" EX 300  
TTL otp:1234  
EXPIRE session:123 1800
```

TTL returns the remaining lifetime. Expiration is especially useful for caches, OTPs, temporary sessions, and short-lived locks.

Hashes

```
HSET user:101 name "Krishna" age 20
HGET user:101 name
HGETALL user:101
```

Lists

```
LPUSH jobs "job1"
RPUSH jobs "job2"
LPOP jobs
LRANGE jobs 0 -1
```

Sets

```
SADD skills Redis NodeJS TypeScript
SISMEMBER skills Redis
SMEMBERS skills
```

Sorted Sets

```
ZADD leaderboard 100 alice
ZADD leaderboard 150 bob
ZREVRANGE leaderboard 0 9 WITHSCORES
```

5. Redis as a Cache

Caching means storing frequently requested data in Redis so the application does not repeatedly query the main database.

Typical flow:

1. Client requests data → 2. Server checks Redis → 3. If cache hit, return Redis data → 4. If cache miss, query database → 5. Store result in Redis with a TTL → 6. Return result.

Example key: product:123

Example value: serialized product JSON

TTL: 300 seconds

Cache-aside pattern is commonly implemented by the application itself.

6. Redis Pub/Sub

Pub/Sub lets publishers send messages to channels. Subscribers listening to the channel receive the messages.

```
Publisher → Channel → Subscribers
```

Use cases include lightweight real-time notifications, broadcasting events, and communication between application instances. Pub/Sub is generally not a durable queue: a subscriber that is disconnected can miss messages.

7. Redis Streams

Streams provide an append-only log of messages/events. They are useful when applications need more structured event processing than basic Pub/Sub provides.

```
XADD notifications * userId 101 message "Welcome"
XRANGE notifications - +
```

Consumer groups allow multiple workers to process stream entries as a coordinated group, which is useful for event-driven architectures.

8. Redis Queues and BullMQ

Redis is frequently used as the storage/backend layer for job queues. **BullMQ** is a popular Node.js queue library that uses Redis to manage jobs, workers, retries, delayed jobs, and related state.

Typical architecture:

```
API Server → Redis/BullMQ → Worker → External service / Database
```

For example, an API can create an email job immediately, while a worker sends the email asynchronously. This keeps the API response fast and allows retries when a job fails.

9. Redis + Socket.IO

Socket.IO provides real-time communication between clients and servers. Redis can help coordinate events when multiple Node.js server instances are running.

A common architecture is: Client → Load Balancer → multiple Node.js servers, with Redis used as shared infrastructure for cross-instance event propagation.

10. Redis in Your NotifyX Project

For an event-driven notification system such as NotifyX, Redis can play several roles:

- **1. BullMQ backend:** store notification jobs and worker state.
- **2. Queuing:** decouple API requests from notification processing.
- **3. Retries:** retry failed notification jobs.
- **4. Delayed jobs:** schedule notifications for later processing.
- **5. Pub/Sub or Socket.IO adapter:** coordinate real-time events across server instances.
- **6. Caching:** cache frequently accessed notification/user information.

Example NotifyX flow

User/API → Express server → BullMQ queue in Redis → Notification worker → Email/SMS/Push provider → status update → Socket.IO → client

11. Redis Persistence

RDB: periodic snapshots of the dataset. It can be efficient and convenient for backups, but data written between snapshots can be lost if Redis fails.

AOF: records write operations so Redis can reconstruct the dataset. It can provide stronger durability depending on the configured fsync policy, with different performance/storage trade-offs.

Redis can also be configured with both approaches. The correct choice depends on the application's durability requirements.

12. Important Concepts

Concept	Meaning
Cache hit	Requested data exists in cache.
Cache miss	Requested data is not in cache; application obtains it elsewhere.
TTL	Time-to-live before a key expires.
Eviction	Redis removes keys when configured memory limits require it.
Persistence	Saving Redis data so it can survive restarts according to configuration.
Atomic operation	A command/operation that executes without another Redis command interleaving inside it.
Pipeline	Send multiple commands together to reduce network round trips.
Transaction	Group commands with MULTI/EXEC semantics.

13. Redis vs PostgreSQL / MongoDB

Redis and a primary database usually serve different roles. PostgreSQL or MongoDB is generally the system of record for persistent application data, while Redis is often used for speed-sensitive temporary/shared state.

Feature	Redis	PostgreSQL / MongoDB
Primary strength	Speed and shared in-memory data	Persistent application data
Typical role	Cache, queue, session, real-time support	System of record
Data model	Key/value + specialized structures	Relational / document
Durability	Configurable persistence	Designed for persistent storage
Best question	Can this data be served/processed very quickly?	What durable business data must we store?

14. Node.js Example

Using the modern Redis Node.js client, an application can connect to Redis and perform basic operations:

```
import { createClient } from "redis";

const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

await redis.set("user:101", JSON.stringify({ name: "Krishna" }));
const value = await redis.get("user:101");

console.log(value);
```

In production, handle connection errors, graceful shutdown, configuration, serialization, and appropriate TTLs.

15. Redis Interview Questions

1. What is Redis and why is it fast?
2. What is the difference between Redis and a traditional database?
3. Explain Redis Strings, Hashes, Lists, Sets, and Sorted Sets.
4. What is TTL and why is it useful?
5. What is a cache hit vs cache miss?
6. Explain the cache-aside pattern.
7. What is Redis Pub/Sub? Is it a durable queue?
8. What are Redis Streams and consumer groups?
9. What is Redis persistence? Explain RDB and AOF.
10. How does BullMQ use Redis?
11. How can Redis support a multi-instance Socket.IO application?
12. What is cache invalidation and why can it be difficult?
13. What are Redis eviction policies?
14. What is a Redis pipeline?
15. When should Redis NOT be used as the primary database?

16. Quick Mental Model

Redis = very fast shared data infrastructure.

Cache → GET/SET + TTL

Counter → INCR/DECR

Object → HASH

Queue → Lists / BullMQ

Events → Pub/Sub / Streams

Ranking → Sorted Sets

Temporary state → TTL

Real-time multi-server coordination → Redis-backed infrastructure

For backend development, focus first on: Strings → Hashes → TTL → caching → Pub/Sub → Streams → BullMQ → persistence → production patterns.